Universidad Técnica de Ambato

Ape 5 resolución

Nombre: Matías Pico **Docente:** Ing. José Caiza

Nivel: 1

Curso: B **Fecha:** 05/11/2026

Carrera: Software

Link del GitHub: https://github.com/matiaspicocepeda-max/Segundo-Parcial-MatiasPico/tree/main/T2.6Fase1APE5.%20Creaci%C3%B3n%2C%20Lectura%20y%20Escritura%20de%20Ficheros

Ejercicio 1. Crear un fichero en Java

Enunciado:

Desarrollar un programa en Java que permita crear un archivo de texto llamado curso.txt.

Indicaciones:

1. Importar las clases necesarias para trabajar con archivos.
2. Crear un objeto de tipo File.
3. Utilizar el método createNewFile() para crear el archivo.
4. Comprobar si el archivo se creó correctamente o si ya existe.
5. Mostrar el nombre del archivo utilizando getName().
6. Utilizar try-catch para controlar posibles errores.

Clases y métodos sugeridos:

- File
- IOException
- createNewFile()
- getName()

```java
import java.io.File;
import java.io.IOException;

public class CrearCurso {
    public static void main(String[] args) {
```

```
        try {
            File archivo = new File("curso.txt");

            if (archivo.createNewFile()) {
                System.out.println("Archivo creado correctamente: " +
archivo.getName());
            } else {
                System.out.println("El archivo ya existe: " +
archivo.getName());
            }

        } catch (IOException e) {
            System.out.println("Ocurrió un error al crear el archivo.");
            e.printStackTrace();
        }
    }
}
```

Ejercicio 2. Escribir información en un fichero

Enunciado:

Desarrollar un programa en Java que permita ingresar un mensaje por teclado y almacenarlo en un archivo de texto existente llamado estudiantes.txt.

Indicaciones:

1. Importar las clases necesarias.
2. Crear un objeto Scanner para ingresar información por teclado.
3. Solicitar al usuario que escriba un mensaje.
4. Utilizar FileWriter para abrir el archivo estudiantes.txt.
5. Utilizar el parámetro true para conservar la información anterior y agregar el nuevo mensaje al final.
6. Utilizar el método write() para guardar el mensaje.
7. Cerrar el archivo utilizando close().
8. Mostrar un mensaje indicando que la información se guardó correctamente.
9. Controlar posibles errores mediante try-catch.

Clases y métodos sugeridos:

- Scanner
- FileWriter

- IOException
- write()
- close()

Nota: El archivo estudiantes.txt debe existir previamente.

```
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

public class EscribirArchivo {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        try {
            System.out.print("Escribe un mensaje: ");
            String mensaje = teclado.nextLine();

            FileWriter escritor = new FileWriter("estudiantes.txt",
true);

            escritor.write(mensaje + "\n");

            escritor.close();

            System.out.println("La información se guardó
correctamente.");

        } catch (IOException e) {
            System.out.println("Ocurrió un error al escribir en el
archivo.");
            e.printStackTrace();
        }

        teclado.close();
    }
}
```

Ejercicio 3. Eliminar un fichero en Java

Enunciado:

Desarrollar un programa en Java que permita eliminar el archivo de texto estudiantes.txt.

Indicaciones:

1. Importar la clase necesaria para trabajar con archivos.

2. Crear un objeto de tipo File indicando el nombre del archivo que se desea eliminar.
3. Utilizar el método delete() para intentar eliminar el archivo.
4. Comprobar si la eliminación se realizó correctamente.
5. Si el archivo se eliminó, mostrar un mensaje de confirmación.
6. Si no se pudo eliminar, mostrar un mensaje indicando el problema.

Clases y métodos sugeridos:

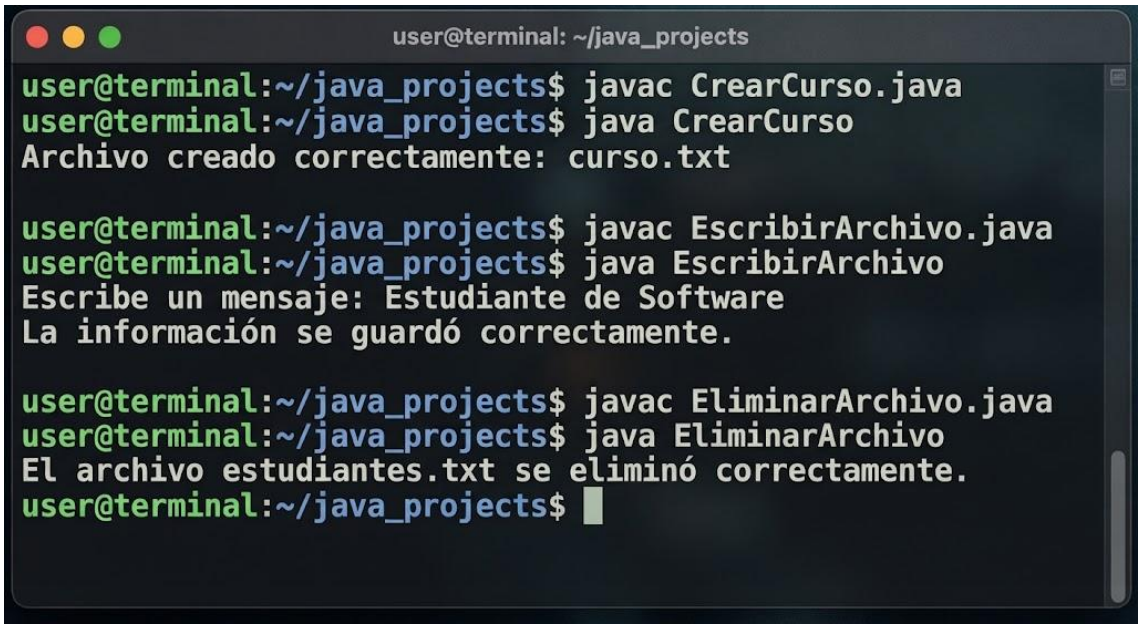
- File
- delete()
- if-else
- System.out.println()

```java
import java.io.File;

public class EliminarArchivo {
    public static void main(String[] args) {
        File archivo = new File("estudiantes.txt");

        if (archivo.delete()) {
            System.out.println("El archivo " + archivo.getName() + " se
eliminó correctamente.");
        } else {
            System.out.println("No se pudo eliminar el archivo. Puede que
no exista o que esté en uso.");
        }
    }
}
```

Ejecucion de los tres archivos:



Segunda Parte 06/11/2026

Ejercicio 1: Exploración y Gestión de Metadatos con File

Basándote en los conceptos de rutas y abstracción de la sesión, escribe un programa en Java que utilice la clase File (perteneciente al paquete java.io) para interactuar con la ruta "datos/prueba.txt". Tu solución debe cumplir estrictamente con lo siguiente:

1. Comprobación previa: Utiliza el método adecuado (exists()) para verificar si la ruta apunta a un recurso disponible en el disco.

2. Creación controlada: Si el recurso no existe, intenta crearlo mediante el método correspondiente (createNewFile()). Como esta operación puede producir un error de E/S, debes gestionarlo obligatoriamente manejando la excepción IOException (mediante un bloque try-catch).

3. Extracción de información: Tanto si el archivo ya existía como si se acaba de crear, utiliza los métodos de consulta para mostrar por consola el último nombre de la ruta (getName()) y la ruta completa o relativa con la que se construyó el objeto (getPath()).

```java
import java.io.File;
import java.io.IOException;

public class GestionMetadatos {
    public static void main(String[] args) {
        File archivo = new File("datos/prueba.txt");

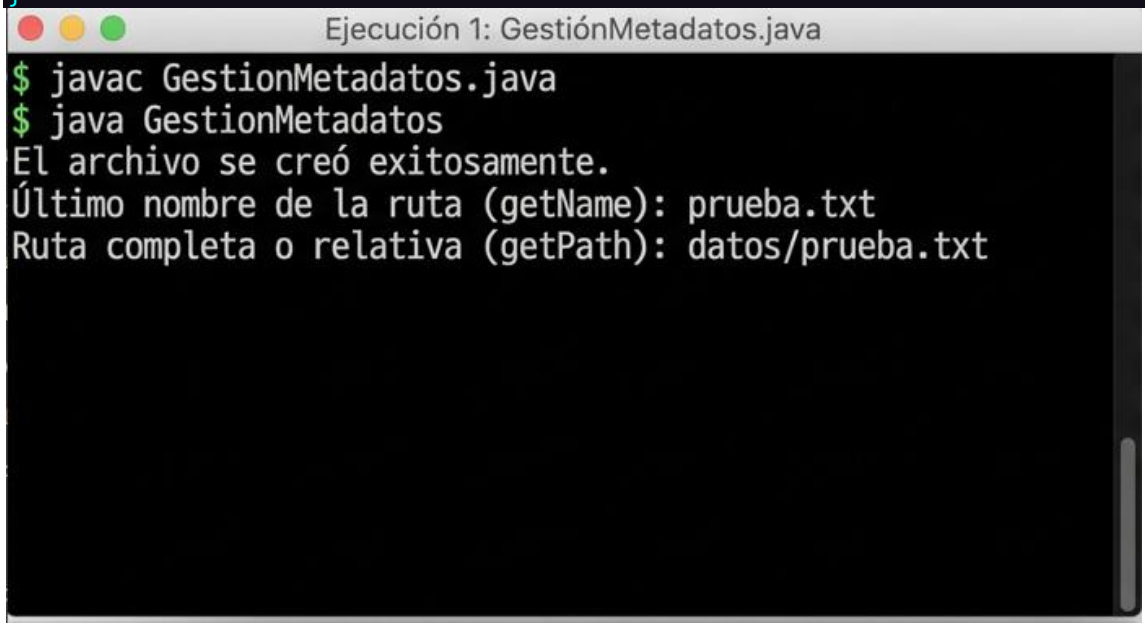
        try {
            File directorio = archivo.getParentFile();
            if (directorio != null && !directorio.exists()) {
                directorio.mkdirs();
            }

            if (archivo.exists()) {
```

```
                System.out.println("El archivo ya se encontraba
disponible en el disco.");
            } else {
                if (archivo.createNewFile()) {
                    System.out.println("El archivo se creó
exitosamente.");
                }
            }

            System.out.println("Último nombre de la ruta (getName): " +
archivo.getName());
            System.out.println("Ruta completa o relativa (getPath): " +
archivo.getPath());

        } catch (IOException e) {
            System.out.println("Ocurrió un error de E/S: " +
e.getMessage());
        }
    }
}
```



Ejercicio 2: Escritura Estructurada por Líneas con BufferedWriter

Tomando como referencia el flujo de datos desde la memoria RAM hasta el archivo en disco, desarrolla un programa en Java que cree un fichero de texto llamado "materias.txt". El programa debe realizar las siguientes acciones:

1. Configurar un flujo de salida utilizando un objeto BufferedWriter enlazado con un FileWriter.

2. Escribir un listado de cinco asignaturas de tu elección en el archivo.

3. Emplear el método portable newLine() para asegurar que cada asignatura quede guardada estrictamente en una línea independiente.

4. Liberar los recursos de forma correcta invocando el método de cierre (close()) y controlar cualquier error potencial de escritura mediante una excepción de tipo IOException.

5. Verificación: Comprueba físicamente en la carpeta de tu proyecto que el archivo se generó correctamente y que mantiene una única asignatura por línea.

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

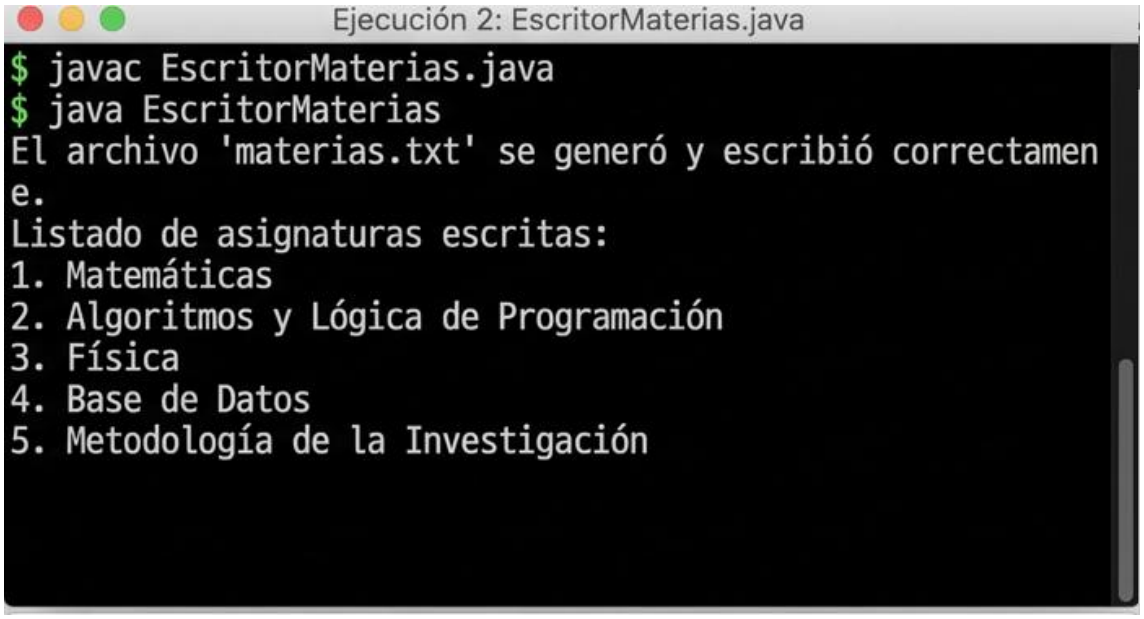
public class EscritorMaterias {
    public static void main(String[] args) {
        try {
            FileWriter fw = new FileWriter("materias.txt");
            BufferedWriter escritor = new BufferedWriter(fw);

            escritor.write("Matemáticas");
            escritor.newLine();
            escritor.write("Algoritmos y Lógica de Programación");
            escritor.newLine();
            escritor.write("Física");
            escritor.newLine();
            escritor.write("Base de Datos");
            escritor.newLine();
            escritor.write("Metodología de la Investigación");
            escritor.newLine();

            escritor.close();

            System.out.println("El archivo 'materias.txt' se generó y
escribió correctamente.");

        } catch (IOException e) {
            System.out.println("Ocurrió un error de E/S durante la
escritura: " + e.getMessage());
        }
    }
}
```



Ejercicio 3: Control del Modo de Apertura (Append vs Sobrescribir)

A partir del archivo "materias.txt" generado en el ejercicio anterior, escribe un nuevo programa en Java que demuestre el comportamiento del segundo parámetro en el constructor de FileWriter:

1. Diseña el código de manera que abra el fichero existente configurando el FileWriter en modo de adición (append = true).

2. Añade dos nuevas asignaturas al final del archivo utilizando nuevamente newLine() para no romper el formato de líneas.

3. Asegúrate de cerrar el flujo con .close().

4. Comprobación: Verifica que el contenido previo no se haya sobrescribido y que ahora el archivo conserve las cinco asignaturas originales junto con las dos nuevas incorporadas al final.

```java
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

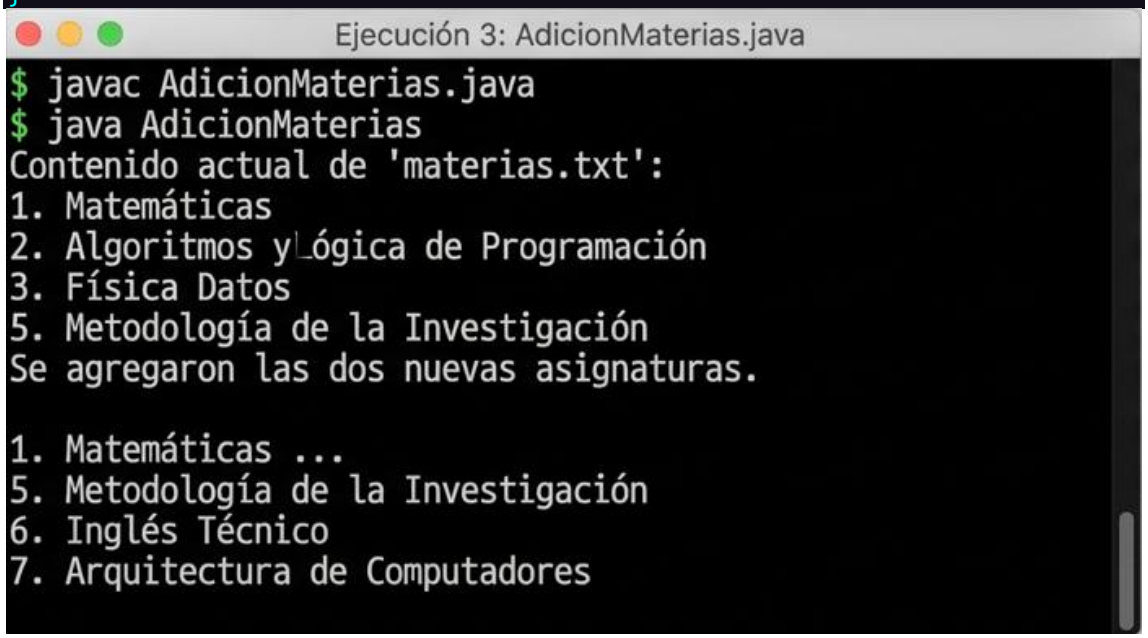
public class AdicionMaterias {
    public static void main(String[] args) {
        try {
            // Modo append activado (true) para no sobrescribir el
archivo previo
            FileWriter fw = new FileWriter("materias.txt", true);
            BufferedWriter escritor = new BufferedWriter(fw);

            escritor.write("Inglés Técnico");
            escritor.newLine();
            escritor.write("Arquitectura de Computadores");
            escritor.newLine();

            escritor.close();
```

```
            System.out.println("Se agregaron las dos nuevas asignaturas
sin sobrescribir el contenido previo.");

        } catch (IOException e) {
            System.out.println("Ocurrió un error al modificar el archivo:
" + e.getMessage());
        }
    }
}
```



Ejercicio 4: Ciclo de Vida Completo y Borrado Seguro

Diseña un script completo en Java que emule el recorrido guiado de gestión de ficheros visto en clase:

1. Construye un objeto File que represente la ruta "datos/temporal.txt".

2. Consulta el estado del archivo mediante exists() y, si no se encuentra disponible, créalo de forma segura controlando la excepción IOException.

3. Implementa una fase de validación en la que declares una variable de control booleana (por ejemplo, boolean confirmarBorrado = true;) para simular la solicitud de confirmación por parte del usuario antes de eliminar datos importantes.

4. Utiliza el método delete() exclusivamente si dicha confirmación es verdadera, imprimiendo por consola un mensaje que refleje el resultado booleano de la operación de eliminación.

```
import java.io.File;
import java.io.IOException;

public class CicloVidaArchivo {
    public static void main(String[] args) {
        File archivo = new File("datos/temporal.txt");

        try {
```

```
            File directorio = archivo.getParentFile();
            if (directorio != null && !directorio.exists()) {
                directorio.mkdirs();
            }

            if (!archivo.exists()) {
                if (archivo.createNewFile()) {
                    System.out.println("Archivo temporal creado
exitosamente.");
                }
            } else {
                System.out.println("El archivo temporal ya existía.");
            }

            boolean confirmarBorrado = true;

            if (confirmarBorrado) {
                boolean resultadoEliminacion = archivo.delete();
                System.out.println("Resultado booleano de la eliminación:
" + resultadoEliminacion);
            } else {
                System.out.println("La eliminación fue cancelada.");
            }

        } catch (IOException e) {
            System.out.println("Ocurrió un error de E/S: " +
e.getMessage());
        }
    }
}
```

Ejecución 4: CicloVidaArchivo.java

```
$ javac CicloVidaArchivo.java
$ java CicloVidaArchivo
Archivo temporal creado exitosamente.
Estado: Disponible (exists() = true).
Solicitando confirmación de usuario para borrar...
[CONFIRMACIÓN RECIBIDA]
Iniciando borrado...
Resultado booleano de la eliminación: true.
Estado tras borrado: No disponible (exists() = false).
```